replica of Catalog Service. You can verify the result with the following command:

```
$ kubectl get pods -l app=catalog-service

NAME                               READY   STATUS    RESTARTS   AGE
catalog-service-68bc5659b8-fkpcv   1/1     Running   0          2s
catalog-service-68bc5659b8-kmwm5   1/1     Running   0          3m94s
```

In the “age” column, you can tell which Pod is the one that has just been deployed to achieve a state with two replicas.

What happens if one of them terminates? Let’s find out. Pick one of the two Pod replicas and copy its name. For example, I might use the Pod named `catalog-service-68bc5659b8-kmwm5`. Then, from a Terminal window, delete that Pod with the following command:

```
$ kubectl delete pod <pod-name>
```

The Deployment manifest declares two replicas as the desired state. Since there is now only one, Kubernetes will immediately step up to ensure the actual state and the desired state are aligned. If you inspect the Pods again with `kubectl get pods -l app=catalog-service`, you will still see two Pods, but one of them has just been created to replace the deleted Pod. You can identify it by checking its age:

```
$ kubectl get pods -l app=catalog-service

NAME                               READY   STATUS    RESTARTS   AGE
catalog-service-68bc5659b8-fkpcv   1/1     Running   0          42s
catalog-service-68bc5659b8-wqchr   1/1     Running   0          3s
```

Under the hood, a ReplicaSet object keeps checking the number of replicas deployed and ensures they are always in the desired state. That’s the basic functionality on top of which you can configure an autoscaler to dynamically